

PROGRAMAÇÃO ORIENTADA A OBJETOS AVANÇADA

PROGRAMAÇÃO E SISTEMAS DE INFORMAÇÃO – 2º ANO CURSO PROFISSIONAL TÉCNICO DE GPSI



“A programação é a arte de organizar e dominar a confusão.”

(Steve Jobs)



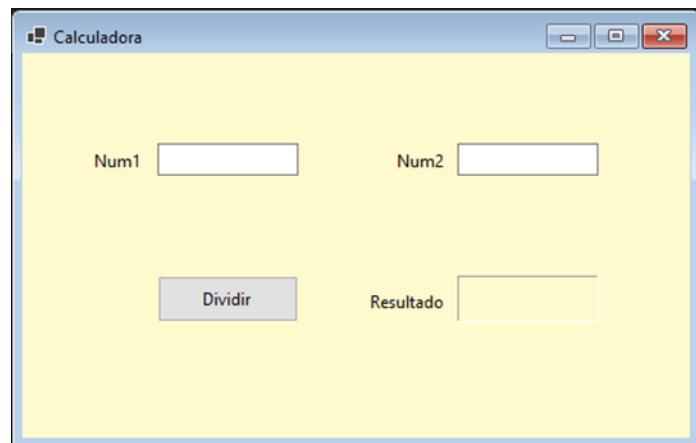
Pensar como objeto: Exceções

Módulo 11 | Linguagem Orientada a Objetos Avançada

Nota: Identifica os projetos pelo número da ficha, do exercício e o teu nome.

Exercício 1

1. Cria uma aplicação simples que possui dois campos de entrada para números e um botão para realizar a divisão entre eles. O resultado será mostrado numa label.
2. Cria um novo projeto Windows Forms e adiciona os seguintes elementos:
 - a. Dois controlos TextBox para entrada dos números.
 - b. Um Button para executar a divisão.
 - c. Uma Label para mostrar o resultado da divisão.
 - d. Um componente ErrorProvider para validação visual dos dados introduzidos.



3. Atribui os nomes que se seguem aos controlos: txtNum1, txtNum2, btnDividir e lblResultado, respetivamente.
4. Cria um método que valide se o divisor é diferente de zero antes de realizar a operação de divisão.
5. Utiliza o componente ErrorProvider para:
 - a. Indicar visualmente campos obrigatórios não preenchidos;
 - b. Assinalar campos que contenham valores inválidos;
 - c. Impedir a execução das operações enquanto existirem erros de validação.
6. No botão dividir, usa blocos try-catch para lidar com exceções que podem ocorrer:
 - a. ao converter os valores das TextBox para números (FormatException);
 - b. ao tentar dividir por zero (DivideByZeroException);
 - c. ao capturar exceções genéricas (Exception) para lidar com outros tipos de erros inesperados;
7. Adiciona botões para efetuar o cálculo das operações soma, subtração e multiplicação. Utiliza os blocos try-catch adequados a cada uma das operações.

Exercício 2

1. Cria um projeto no Windows Forms para calcular a média de notas de um aluno. Este exercício permitirá praticar os conceitos de programação orientada a objetos (POO) e blocos try-catch para tratamento de exceções.
2. Cria um formulário com:
 - a. Duas TextBox para o utilizador inserir as notas.
 - b. Um Button para calcular a média.
 - c. Uma Label para mostrar o resultado da média.
 - d. Um componente ErrorProvider para validação visual dos dados introduzidos.
3. Quando o utilizador clicar no botão "Calcular Média", o programa deverá tentar calcular a média das notas inseridas.
4. Utiliza o componente ErrorProvider para:
 - a. Indicar visualmente campos obrigatórios não preenchidos;
 - b. Assinalar campos que contenham valores inválidos;
 - c. Impedir a execução das operações enquanto existirem erros de validação.
5. Caso ocorra um erro (por exemplo, se o utilizador inserir uma nota inválida), uma mensagem de erro deve ser exibida ao utilizador (FormatException), ou qualquer outra exceção (Exception ex).

Exercício 3

1. Cria um projeto Windows Forms que represente um carro e que permita ao utilizador inserir dados sobre o carro, como modelo, ano e preço, e mostrar numa mensagem algumas informações sobre o carro, como modelo, ano e preço.
2. Cria a Classe Carro com as propriedades como modelo (string), ano (int) e preço (decimal/double).
3. Cria a interface gráfica e adiciona os seguintes elementos:
 - a. Três TextBox para inserir o modelo, o ano e o preço do carro;
 - b. Um Button para mostrar as informações do carro numa caixa de mensagem
 - c. Um componente ErrorProvider para validação visual dos dados introduzidos.
4. Utiliza o componente ErrorProvider para:
 - a. Antes de criar o objeto Carro, indicar visualmente campos obrigatórios não preenchidos;
 - b. Assinalar campos que contenham valores inválidos;
5. Se houver um erro de formato ao converter os valores dos TextBox para os tipos esperados (FormatException), ou qualquer outra exceção, uma mensagem de erro será mostrada ao utilizador.

BOM TRABALHO! TU ÉS CAPAZ! CONSTRÓI O TEU CONHECIMENTO...

Os facilitadores da aprendizagem,

Andreia Quintal | Carlos Almeida | Cristina Gonçalves | Pedro Cardoso